DeepSeek-V4.1-Flash 架构：技术报告/源码解析

> 以下部分链接需要《大模型》笔记访问权限

📌
论文：《DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression》
模型仓库：deepseek-ai/DeepSeek-V4.1-Flash
推荐阅读：
🐞🐞🐞🐞🐞 DeepSeek-V4 架构：技术报告/源码解析
🐞🐞🐞🐞🐞 mHC：Manifold-Constrained Hyper-Connections
🐞🐞🐞🐞 Engram：Conditional Memory via Scalable Lookup
🐞🐞🐞 Muon 优化器
🐞🐞 DeepSeek V4 多模态：Thinking with Visual Primitives

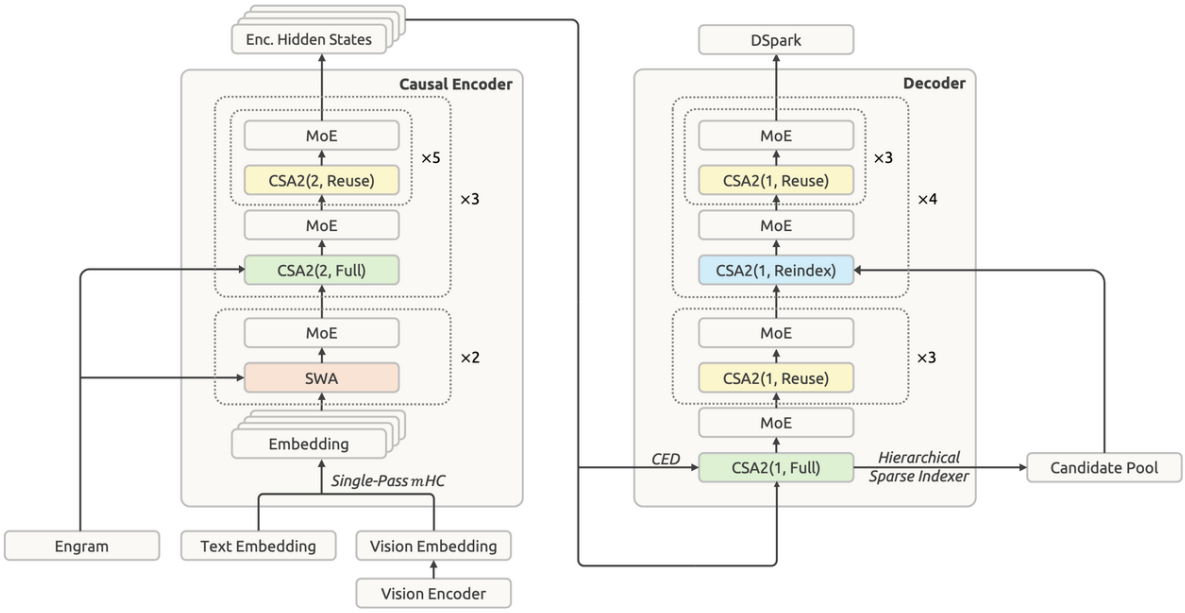
🤔
DeepSeek-V4.1-Flash 相对 DeepSeek-V4 的核心改进：

- **CED（Causal Encoder-Decoder）**：语言模型主干包括 20 层 Causal Encoder + 20 层 Decoder，Decoder 的全局 KV 由 Causal Encoder 最后一层 $H_{L/2}$ 投影得到，Prefill 的 FLOPs 大致减半。
- **CSA2（Compressed Sparse Attention 2）**：简化了 DeepSeek-V4 CSA 的 KV 压缩建模，并且通过 Full / Reindex / Reuse 三种模式跨层共享 main KV / indexer K / Top-K Indices。
- **Hierarchical Sparse Indexer**：Decoder 的第一个 Full 层额外做 block-wise 候选池选择，Decoder 第二层开始的 Reindex 层只在候选池里打分，深层 Indexer 打分计算量不再随上下文增长。
- **Engram**：Engram 以每个位置结尾的 n-gram 为 Key 查询嵌入表，取回的记忆向量经门控写回残差流，本质上是静态知识的检索，不占用 Attention 与 FFN 的计算。
- **Single-Pass mHC、DSpark、head-wise Muon、FP4 main KV cache**：mHC 消除 kernel 间依赖，投机解码升级为 DSpark。

对比参数/上下文规格：V4-Flash 是 284B-A13B，V4-Pro 是 1.6T-A49B，V4.1-Flash 是 552B-A8B(prefill)/A16B(decode)/196B(Engram)，均支持 1M 上下文。

> 196B 的 Engram 不计入总参数，推理时靠 RDMA 从 host memory 预取。

参考DeepSeek-V4 架构：技术报告/源码解析，DeepSeek V4 Flash 的模型层注意力架构是 CSA（Compressed Sparse Attention）+ HCA（Heavily Compressed Attention）交错分布（前两层是 SWA）。DeepSeek-V4.1-Flash 所有层替换成统一 CSA2 架构（保留前两层 SWA）。



上图是 DeepSeek-V4.1-Flash 架构，Attention 除了前 2 层的 SWA，其余层统一为 `CSA2(ratio, mode)`，`ratio` 和 `mode` 两个静态参数直接决定层性质，也是下面的关键内容：`ratio` 来自 `compress_ratios`，`mode` 由 `kv_source_layer_ids` 与 `index_source_layer_ids` 共同决定。

1. CED（Causal Encoder-Decoder）：解码器全局 KV 由编码器最后一层投影

动机： Agentic 场景工具调用频繁，每轮都产生新的 prefill 请求，KV cache miss 造成极高 TTFT。DeepSeek V4 的 prefill 复杂度是 $O(NL)$，N 为序列长度、L 为层数，在 Input-Heavy 负载下是主要成本。

实现： 做法类似《You Only Cache Once: Decoder-Decoder Architectures for Language Models》（YOCO）的策略，让模型后半层共享前半层 KV cache，DeepSeek V4.1-Flash 的 CED（Causal Encoder-Decoder）在 YOCO 上做两处增强：

- 全局 KV 从 Encoder 输出投影：给每个解码器层配独立投影权重，从共享 hideen states 中独立投影。
- 保留逐层的局部注意力以维持局部 KV 的计算深度。

1.1 全局 KV 从 Encoder 输出投影

把 Transformer 的前 $L/2$ 层作为 Causal Encoder，对后 $L/2$ 层解码器，全局 KV 不由本层 hidden state H_l 得到，而是从第 $L/2$ 层输出的 hidden state $H_{L/2}$ 直接投影：

$$C_l = H_{L/2} W_l^{KV}, \quad Z_l = H_{L/2} W_l^{Z}, \quad l > \frac{L}{2}$$

- $H_{L/2} \in \mathbb{R}^{N \times d}$ 是 Causal Encoder 最后一层的输出，d 为隐藏维度（5120）。
- C_l 是第 l 层的 KV 条目，Z_l 是对应的压缩权重。
- W_l^{KV}, W_l^{Z} 是层相关的投影权重，各解码器层拿到的仍是各自的 KV。

因此 Prefill 的 FLOPs 大约可以减半：在拿到 $H_{L/2}$ 之后，后 $L/2$ 层的全局 KV 通过两次矩阵乘补齐，不需要经过后 $L/2$ 层的完整计算（Attention、MoE 等）。

> 关键注意点：虽然上面的公式（来自 DeepSeek V4.1 Flash 原论文）是每个解码器层都有自己的 W_l^{KV}，但是 CSA2 的 Reuse 模式（下文会介绍）直接共享 Main KV，因此实际只实例化一份全局 KV，解码器层直接共享。

1.2 Decoder SWA Bounded Replay

DeepSeek-V4.1-Flash 的 CSA2 是由 DeepSeek V4 的 CSA 简化而来的，CSA 下文会介绍。CSA2 仍然保留了 Sliding Window Attention 的计算细节：

> 📌
> 每层保留 `n_win=128` 个 uncompressed KV，跟压缩 KV 拼在一起做 attention：
> **这其实是对 CSA/HCA 的漏洞修复，有两个动机：**
>
> 1. 维持严格因果性带来的块内盲区：CSA/HCA 的 query 只能 select " 已经完成的" 压缩块，跟 query 同属一个压缩块的 m-1 个邻居彼此看不见（CSA 的 m=4、HCA 的 m'=128 都有这个问题）。
> 2. 最近邻优先：最近 token 相关性最强，留未压缩的精确版本比压缩后的更精确。

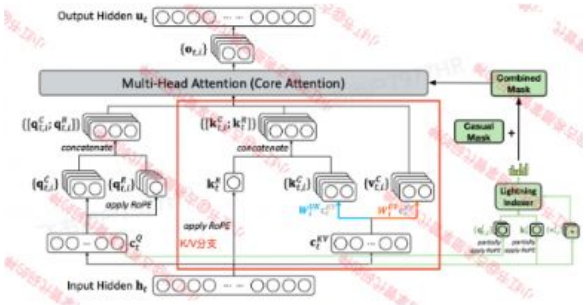
而局部注意力 SWA 的逻辑不是上面的共享全局 KV，SWA 的 key 与 value 都从本层的 H_l 直接计算，但是代价就是 prefill 阶段解码器仍需多处理 $n_{\text{win}} \times L/2$ 个 Token（$L/2$ 层，每层状态最多向前拓展 n_{win} 个 Token）才能补齐完整的 SWA 状态，多轮短提示词场景下这笔开销不可忽略。

但是 DeepSeek V4.1 Flash 消融观察到 SWA 的实际有效感受野远小于理论上的 $n_{\text{win}} \times L/2$，并且模型退化可以忽略，因此 V4.1 Flash 据此提出 Decoder SWA Bounded Replay：只重放提示词最后 n_{win} 个 token 做近似重建，因此 $N \gg n_{\text{win}}$ 时 prefill 复杂度从 $O(NL)$ 降到 $O(NL/2 + n_{\text{win}} \times L/2) \approx O(NL/2)$。

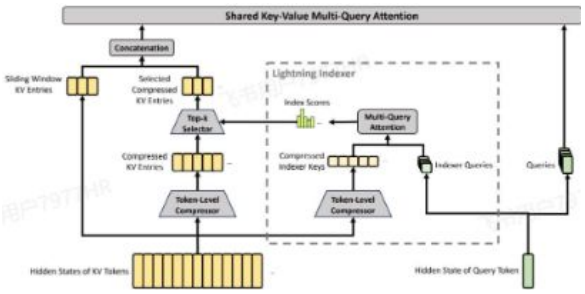
2. CSA2（Compressed Sparse Attention 2）

2.1 回顾第一代 CSA（Compressed Sparse Attention）

首先简要回顾 DeepSeek-V4 架构：技术报告/源码解析中 CSA 的架构：



DS V3.2 的 DSA



DS V4 的 Compressed Sparse Attention

DeepSeek V3.2 的 DSA 直接在原始 KV 上用 Lightning Indexer 做 top-K Selection，每个 query 的 KV 候选大小仍是$O(n)$。V4 先进行 KV Sparse 压缩到$O(n/m)$再进行 Selection，因此 indexer 排序候选集变为之前的$1/m$。分两步实现，先 KV Compress 把$O(n)$序列压缩到$O(n/m)$，后 Lightning Indexer 进行 Top-K 选择。

KV Compress

输入 $H \in \mathbb{R}^{n \times d}$，首先用两个可学习的 KV projection $W_a^{KV}, W_b^{KV} \in \mathbb{R}^{d \times c}$ 映射到 Latent（这里和 MLA 的逻辑是一样的），但是需要注意这里映射了 2 个 Latent 表示为的是压缩的时候进行窗口平滑（Sliding Window）：

$$C^a = H \cdot W_a^{KV}, C^b = H \cdot W_b^{KV}$$

此外，需要对应的另外两个可学习的 KV projection $W_a^Z, W_b^Z \in \mathbb{R}^{d \times c}$ 映射另外一组 Gate Latent，映射维度和上面一样，用来计算压缩时候的加权率（Gating）：

$$Z^a = H \cdot W_a^Z, Z^b = H \cdot W_b^Z$$

然后 m 个连续 token 计算两个压缩加权比率（论文里面取$m = 4$）：

$$[S^a_{mi:m(i+1)-1}; S^b_{m(i-1):mi-1}] = \text{Softmax}_{\text{row}}\Big([Z^a_{mi:m(i+1)-1} + B^a;\ Z^b_{m(i-1):mi-1} + B^b]\Big)$$

用上面的加权比率就可以将 m 个连续 token 合成一个 compressed entry C^{Comp}_i，注意 C^a 用当前窗口 $[mi, m(i+1)-1]$，C^b 用**前一个**窗口 $[m(i-1), mi-1]$，两个窗口叠加，这是 CSA 平滑的关键：

$$C^{\text{Comp}}_i = \sum_{j=mi}^{m(i+1)-1} S^a_j \odot C^a_j + \sum_{j=m(i-1)}^{mi-1} S^b_j \odot C^b_j$$

- $B^a, B^b \in \mathbb{R}^{m\times c}$：可学习的位置偏置（相当于滑动窗口的 Absolute Position Embedding）。
- $\odot$：Hadamard 积，即对应位置相乘，结果 shape 不变。

Lightning Indexer

对 query token t，先生成低秩 Latent 再生成 indexer queries（本质上仍然是 MLA）：

$$\mathbf{c}^Q_t = \mathbf{h}_t W^{DQ}$$

$$[\mathbf{q}^I_{t,1}; \ldots; \mathbf{q}^I_{t,n^I_h}] = \mathbf{q}^I_t = \mathbf{c}^Q_t W^{IUQ}$$

同时需要生成 query head 之间的加权权重（n^I_h 表示 Indexer 的 query 的 head 数量）：

$$[w^I_{t,1}; w^I_{t,2}; \ldots; w^I_{t,n^I_h}] = \mathbf{w}^I_t = \mathbf{h}_t \cdot W^w,$$

然后应用 ReLU 风格的 index score（需要在多 head 之间求和）：

$$I_{t,s} = \sum_{h=1}^{n^I_h} w^I_{t,h} \cdot \text{ReLU}(\mathbf{q}^I_{t,h} \cdot K^{\text{IComp}}_s)$$

Top-k 选择：$C^{\text{SprsComp}}_t = \Big\{C^{\text{Comp}}_s \mid I_{t,s} \in \text{Top-k}(I_{t,:})\Big\}$。

Shared KV MQA

选出 top-k compressed entries 之后做 MQA：

$$[\mathbf{q}_{t,1}; \ldots; \mathbf{q}_{t,n_h}] = \mathbf{q}_t = \mathbf{c}^Q_t W^{UQ}$$

$$\mathbf{o}_{t,i} = \text{CoreAttn}(\text{query} = \mathbf{q}_{t,i}, \quad \text{key} = C^{\text{SprsComp}}_t,\ \text{value} = C^{\text{SprsComp}}_t)$$

最后把多个 head 的 $\mathbf{o}_{t,i}$ 拼起来得到 $\mathbf{o}_t = [\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \ldots; \mathbf{o}_{t,n_h}] \in \mathbb{R}^{cn_h}$。

Grouped Output Projection

上一步合并的 $\mathbf{o}_t$ 的 hidden dim cn_h 非常大，如果直接进行 w_o 投影，投影矩阵是 $cn_h \times d$，参数量和计算量都很夸张。因此先分组降维，再合并投影。

- **分组**：把 n_h 个头平均分成 g 组，每组有 n_h/g 个头。每组拼起来的向量是: $\mathbf{o}^G_{t,i} \in \mathbb{R}^{c\cdot n_h/g}$
- **组内降维**：每一组各自用一个小的投影矩阵，把 $c \cdot n_h/g$ 维压到 d_g 维:$\mathbf{o}^G_{t,i} W_g^{\text{o_a}} = \mathbf{o}^{G'}_{t,i} \in \mathbb{R}^{d_g}$
- **合并后再投影到 d**：把 g 个降维后的小向量拼起来 (总共 $d_g \cdot g$ 维)，再用一个投影矩阵送到最终的 d 维: $\mathbf{o}^{G'}_{t,i} = \widehat{\mathbf{o}}_t W^{\text{o_b}} \in \mathbb{R}^d$

2.2 CSA2 相比 CSA 进行简化

动机： DeepSeek V4 的第一代 CSA 的设计推高了实现复杂度与训练开销：

- 在压缩率 m 设置下，每条 main KV 由 $2m$ 条原始 KV 生成，即 $C^a = H \cdot W^{KV}_a, C^b = H \cdot W^{KV}_b$。
- 压缩时带可学习的绝对位置偏置 B^a, B^b，用来编码这 $2m$ 个位置。
- indexer 的 Key 从 hidden state 独立压缩。

实现： CSA2 采用更简单的 KV 压缩建模，直接退化成单个窗口内的 softmax 池化：

$$C = HW^{KV}, Z = HW^Z$$

$$S_{mi:m(i+1)-1} = \text{Softmax}_{\text{row}}\left(Z_{mi:m(i+1)-1}\right),$$
$$C^{\text{Comp}}_i = \text{RMSNorm}\left(\sum_{j=mi}^{m(i+1)-1} S_j \odot C_j\right)$$

- $H \in \mathbb{R}^{N\times d}$ 是当前层输入，解码器层为 $H_{L/2}$。
- C^{Comp}_i 是第 i 条压缩 KV 条目，序列长度是原本的 $1/m$。

对照 DeepSeek-V4 架构：技术报告/源码解析 里 CSA 的同一组公式，关键差别在于取消了可学习的绝对位置偏置 B^a、B^b 与错位叠加，增加了一个 RMSNorm。对应到实现代码 `Compressor.forward`：

对于 indexer K 的简化：CSA2 直接把 main KV 条目投影成 indexer K，$K^I_s = \text{RMSNorm}(C^{\text{Comp}}_s W^{IK})$，不再单独从 hidden state 压缩：

Indexer 打分的形式和 DeepSeek V4 一致，即 ReLU 风格的 index score：

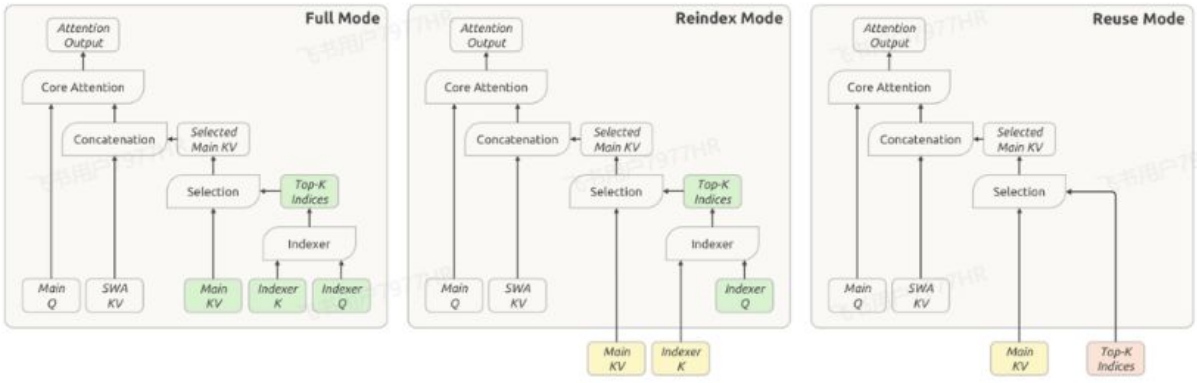
$$I_{t,s} = \sum_h w^I_{t,h} \cdot \text{ReLU}(\mathbf{q}^I_{t,h} \cdot K^I_s)$$

> GLM-5.3-Flash 同样也是用了同一机制。

但是相比 DeepSeek V4 缩放常数不同：V4 是 $(d^I_h)^{-1/2}$，V4.1 是 $(d^I_h)^{-1/2}(n^I_h)^{-1/2}$，分摊 head 数对分数的影响。

2.3 CSA2 的 Full / Reindex / Reuse 模式

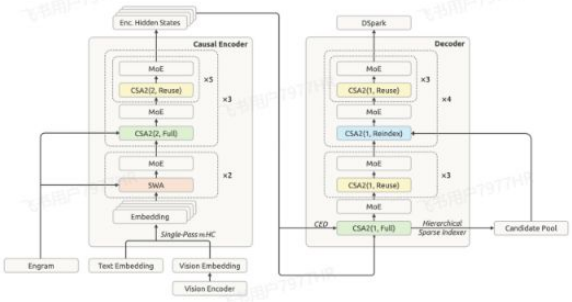
CSA2 层可以被设置成 Full / Reindex / Reuse 三种静态模式。Full / Reindex / Reuse 三种静态模式共有的部分是 main Q 和自己的 SWA KV，区别在于 main KV、indexer K、Top-K Indeices 的来源。



- **Full**：即默认的 CSA 计算模式，main KV、indexer K、Top-K Indeices 均由当前层重新计算，即压缩 main KV、投影 indexer K、执行 indexer 得到新的 Top-K Indeices。
- **Reindex**：main KV 与 indexer K 复用最近一个 Full 层，但用本层的 indexer Q 重新打分选出当前层的 Top-K Indeices，稀疏选择因此能逐层变化并且保持 cache 共享。
- **Reuse**：main KV 与 Top-K indexer 都复用，因此不必计算 indexer Q / indexer K。

在 1M 量级的长文本注意力下，两个关键的开销是`计算`和`缓存`，共享 main KV 与 indexer K 目标是缓解 cache 存储压力，复用 Top-K 缓解的是 indexer 计算压力。Reindex 是`计算`和`缓存`平衡的中间，保住存储共享的同时保留层间 Top-K 选择差异，Reuse 则类似 GLM-5.2 的 IndexShare，跨层复用稀疏选择。

2.4 层级调度：从 kv_source 与 index_source 列表推导





DeepSeek-V4.1-Flash 的层级调度（右上图）是非常干净的 CSA2（除了前两层是 SWA），相比于 DeepSeek V4 的 1:1 CSA + HCA 混合注意力看起来统一了很多，但是实质上 DeepSeek-V4.1-Flash 是通过两个维度去控制层注意力的模型，即上文 CSA2 的压缩率 m 以及 Full / Reindex / Reuse 模式，参考左上图，`CSA2 (_, _)` 的两个静态参数决定层的模式，层模式在 config.json 里没有显式字段，靠两个列表交叉判定：

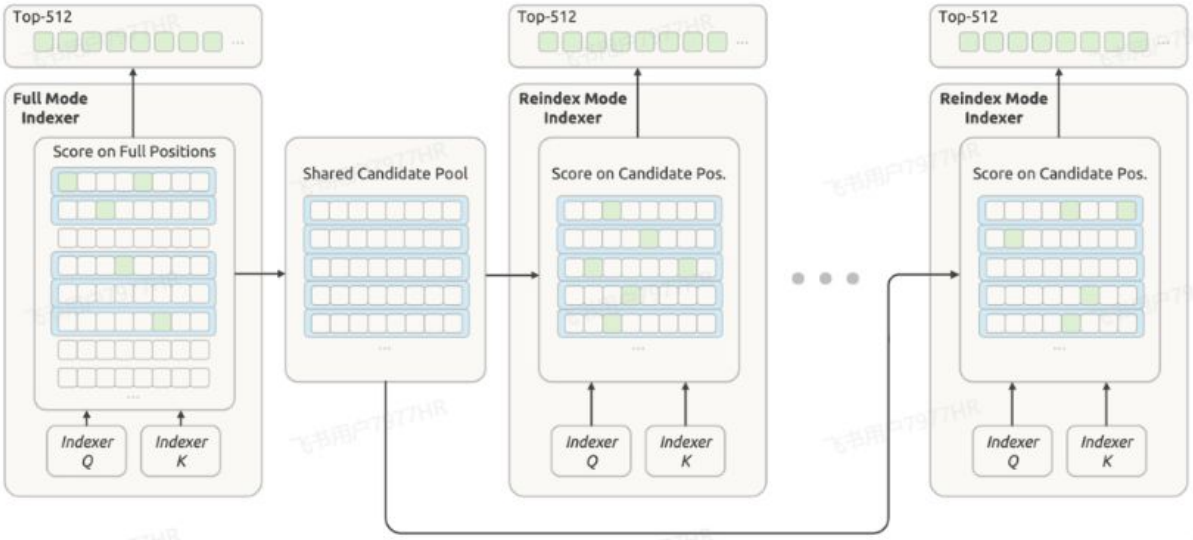
层模式判断依据：既是 KV 源又是 index 源的设置为 Full，只是 index 源的设置 Reindex，两者都不是设置 Reuse。

架构层位置	层号	压缩率（`ratio`）	模式（`mode`）
Encoder 前两层	0, 1	0	SWA
Encoder 2 / 8 / 14 层	2 / 8 / 14	2	Full
Encoder 其余	总计 15 层	2	Reuse
Decoder 第一层	20	1	Full
Decoder 21 / 22 / 23 层	21 / 22 / 23	1	Reuse
Decoder 24 / 28 / 32 / 36 层	24 / 28 / 32 / 36	1	Reindex
Decoder 其余	总计 12 层	1	Reuse

2.5 Hierarchical Sparse Indexer

动机： 跨层索引复用（CSA2 的 Reuse）减少了 indexer 的执行次数，但剩下的其他层的 Indexer 仍要对全部因果可见位置打分，计算压力随上下文线性增长，在 1M 上下文时重新变成主要瓶颈。

实现： 解码器第一个 Full 层（第 20 层）除给选择当前层 Top-K，还额外做一次块级选择：每块取块内位置的最高 index score 作为这个块的分数，选出分数最高的若干块，这些块覆盖的位置作为候选池供后面的 Reindex 层复用。



这个过程可以很直观的被上面的图示表示，公式可以表述为：

$$\widetilde{I}_{t,b} = \max_{s \in b} I_{t,s}, \quad \mathcal{B}_t = \text{Top-}k_{\text{blk}}\left(\widetilde{I}_{t,:}\right)$$

- b 是块索引，$\widetilde{I}_{t,b}$ 是查询 t 对块 b 的分数。
- $\mathcal{B}_t$ 是选中的块集合，Top-K 大小为 `candidate_topk_blocks`。

因此，Hierarchical Sparse Indexer 把深层 indexer 的每 query 打分量限制在一个固定上限内，降低了计算量：

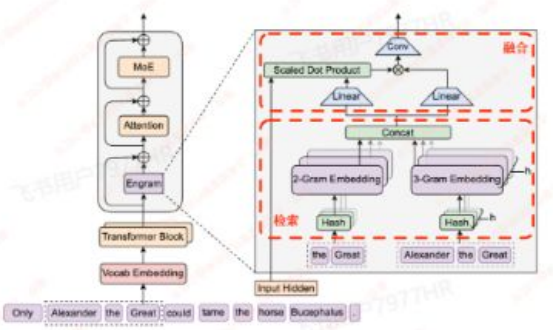
> 关键细节：
> - 最新的块还没被填充完，被显式设置 $+\infty$ 强制保留。
> - Hierarchical Sparse Indexer 在后训练阶段引入，因此训推不一致的问题被后训练缓解。

Hierarchical Sparse Indexer 带来的问题：

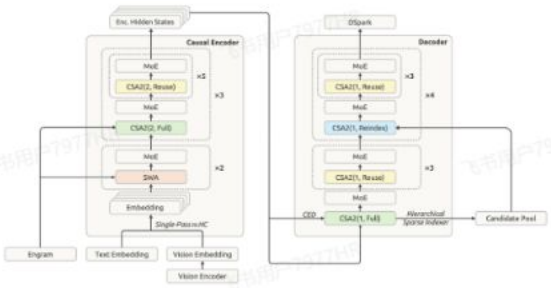
- 第 20 层（Decoder 第一个 Full 层）仍要全范围扫描。
- 候选池由第 20 层（Full）单独决定，深层 Selection 被这一层的判断限制，可能带来退化。

3. Engram

> 前置阅读：Engram：Conditional Memory via Scalable Lookup，Engram 是 DeepSeek 提出的模块，这次 V4.1-Flash 集成到了模型中，之前 Qwen3.8-Flash-Next 也集成了 Engram 技术。



Engram 原文架构示意图



DeepSeek V4.1 Flash 架构

Engram：在标准的 Transformer 主干结构上增加 Engram 条件记忆模块，通过残差连接将 Engram 串联到 Transformer-Block 主干网络：$H^{(\ell)} \leftarrow H^{(\ell)} + Y$，注意模型结构中只有部分 Transformer-Block 增加 Engram 模块。

增加 Engram 模块后，在架构上静态模式存储（Engram）与动态计算（Attention、FFN）分离。

新的模型结构处理每一个位置t的 token 方法分为两个阶段：给定一个输入序列 $X = (x_1, ..., x_T)$ 以及第 ℓ 层的隐藏状态 $H^{(\ell)} \in \mathbb{R}^{T \times d}$，Engram 模块通过检索和融合两个阶段处理每个位置 t 的特征：

- **检索：**把后缀 n-gram 的 token id 压缩成规范 id，经多头哈希查嵌入表，得到记忆向量 e_t。
- **融合：**以隐藏状态 h_t 为 query、$W_K e_t$ 为 key 算出门控 α_t，对 value $W_V e_t$ 加权后写回。原论文在写回前还经过一层短卷积，这里 DeepSeek-V4.1-Flash 相对原始 Engram 设计去掉了短因果卷积，理由是收益不足以抵消推理栈上的复杂度。

3.1 两个 Engram 模块的位置

Engram 以每个位置结尾的 n-gram 为 Key 查询嵌入表，取回的记忆向量经门控写回残差流，本质上是静态知识的检索，因此不再占用 Attention 与 FFN 的计算。

V4.1-Flash 在第 1 层和第 14 层（层计数从 0 开始）各放一个 Engram 模块，合计 196B 参数（196B 的 Engram 参数均分给两个模块），不计入 552B 主干网络参数。选择这两个层插入位置的原因是让显存占用在训练流水线各段间均衡。每个模块用 $\{2, 3, 4\}$ 三阶 n-gram、每阶 8 个 hash head、每阶总嵌入维度 2048（`engram_head_dim` 乘 `engram_n_heads` 是 256 x 8 = 2048）：

这部分参数按 n-gram 命中稀疏访问，逐 token 计算几乎不增加，代价是嵌入表放不进显存，需要依靠确定性寻址从主机内存预取。

3.2 检索：多头哈希查表

下面直接复用Engram：Conditional Memory via Scalable Lookup中的符号表示：

- $g_{t,n}$ 是以位置 t 结尾、长度为 n 的 n-gram，token id 已经过分词器压缩。
- $\phi_{n,k}$ 是第 n 阶第 k 个头的哈希函数，$z_{t,n,k}$ 是哈希得到的行号。
- $E_{n,k}$ 是对应的嵌入表，e_t 是所有头拼接后的记忆向量。

检索阶段要解决的问题：n-gram 的组合数随阶数指数增长，为每个组合简历 Embedding 参数量会造成爆炸。

解决方法：

1. **哈希降维（解决组合参数爆炸）**：$g_{t,n}$ 经 $\phi_{n,k}$ 映射到固定大小表内的行号，查表后拼接：$z_{t,n,k} \triangleq \phi_{n,k}(g_{t,n})$，$e_{t,n,k} = E_{n,k}[z_{t,n,k}]$，$e_t \triangleq \|_{n=2}^{N} \|_{k=1}^{K} e_{t,n,k}$
2. **多头（解决哈希冲突）**：每阶用 K 个头，各自使用一张表长为不同质数的表，减少两个不同的 n-gram 在全部 K 个表上完全一致（冲突）的可能性。

> V4.1-Flash 的配置：
>
> - $N = 4$（`engram_max_ngram_size`），n 取 2、3、4 三阶，原论文 Engram-27B 是 $N = 3$。
> - $K = 8$（`engram_n_heads`），每个模块共 $(N-1) \times K = 3 \times 8 = 24$ 张表。
> - $e_{t,n,k} \in \mathbb{R}^{256}$（`engram_head_dim`），每阶 $8 \times 256 = 2048$ 维，三阶拼接后 $e_t \in \mathbb{R}^{6144}$。
> - 分词器压缩后剩 99092 个规范 id（`engram_compressed_vocab_size`），规范化参考 Engram：Conditional Memory via Scalable Lookup。

$\phi_{n,k}$ 的实现与原 Engram 相同：各阶规范 id 乘上本层的奇数乘子，逐位异或，再对第 k 个头的质数表长取模。

V4.1 实现可以从参考`NgramHashState.forward`：

表长的取法：每个头的表长是互不重复的质数，从 `engram_vocab_size` 减 1 开始用 `find_next_prime` 逐个取下一个未用过的质数，每张约 16M 行，对应 `EngramLayout.from_args`：

两个模块共 48 张表，按模块求和就是 `engram_num_embeddings` 的 384006168 与 384016682。合计 768022850 行，每行 256 维，约 196.6B 参数，即论文中的 196B。

3.3 融合：多分支门控写回

> 单条残差流的门控可以写成 h_t 与 key 的归一化内积后 sigmoid：$\alpha_t = \sigma\left(\frac{\mathrm{RMSNorm}(h_t)^\top \mathrm{RMSNorm}(k_t)}{\sqrt{d}}\right)$，最终输出为：$\tilde{v}_t = \alpha_t \cdot v_t$。

DeepSeek-V4.1-Flash 的残差流经 mHC 加宽成 $M = 4$ 路（`hc_mult`），使用 Engram：Conditional Memory via Scalable Lookup中多分支集成的版本（对照上面的单条残差流的门控理解）：

$$\alpha_t^{(m)} = \sigma\left(\frac{\mathrm{RMSNorm}(\mathbf{h}_t^{(m)})^\top \mathrm{RMSNorm}(\mathbf{W}_K^{(m)}\mathbf{e}_t)}{\sqrt{d}}\right), \quad \mathbf{u}_t^{(m)} = \alpha_t^{(m)} \cdot (\mathbf{W}_V \mathbf{e}_t)$$

- $\mathbf{h}_t^{(m)}$ 是第 m 条残差流在位置 t 的隐藏状态，$d = 5120$。
- $\mathbf{W}_K^{(m)}$ 每条流一份，每条流各自判断是否写入这条记忆。
- $\mathbf{W}_V$ 四条流共用，写入的内容是同一份。

> V4.1 工程上把 1 个 $\mathbf{W}_V$ 与 M 个 $\mathbf{W}_K^{(m)}$ 合并成`wkv`，单次矩阵乘法获取 `key` 与 `value`。

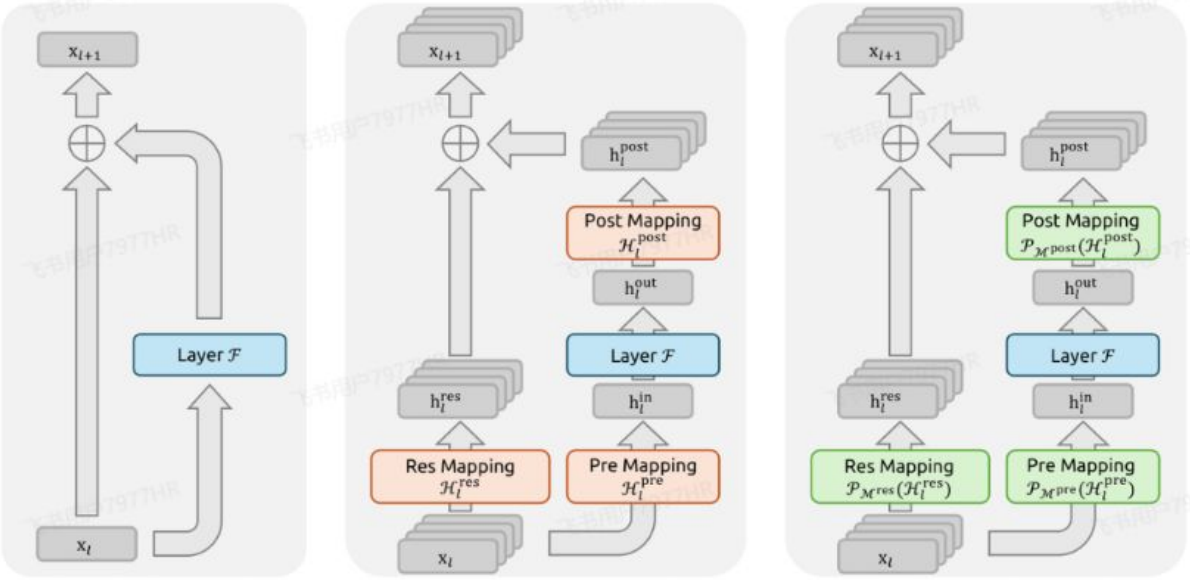
对应到 `Engram.forward`：先查表拼接得到 $\mathbf{e}_t$，经 `wkv` 切出 `key` 与 `value`，按流算归一化内积与 gate，最后每条流加上同一份 value 乘 gate：

需要注意的两个细节：

- 两个 RMSNorm 的缩放权重 `q_weight` 与 `k_weight` 在内积中逐元素相乘。
- DeepSeek-V4.1-Flash 是多模态大模型，图像输入的 token 位置经 `token_mask` 把 gate 置零，恒等通过。

4. 其他细节改动

4.1 Single-Pass mHC 与 Mega-mHC



(a) Residual Connection (b) Hyper-Connections (HC) (c) Manifold-Constrained HC (*m*HC)

动机： mHC 把残差流拆成 n 份并行副本，块间更新式是 $\mathbf{x}_{l+1} = \mathcal{H}_l^{res}\mathbf{x}_l + \mathcal{H}_l^{post\top}\mathcal{F}(\mathcal{H}_l^{pre}\mathbf{x}_l, \mathcal{W}_l)$，$(\mathcal{H}_l^{pre}, \mathcal{H}_l^{res}, \mathcal{H}_l^{post}) = \mathcal{H}(X_l)$。理想情况下中间激活访存下界为 $(2n+2)d$，而 DeepSeek V4 的实现是三个串行 kernel，因为 $\hat{\mathbf{x}}_l = \mathcal{H}_l^{pre}\mathbf{x}_l$ 要等 $\mathcal{H}(X_l)$ 在隐藏维上求和完成，合计 $(4n+4)d$，开销是下界的两倍。

对应到 mHC：Manifold-Constrained Hyper-Connections的符号系统，每个子层的更新式为 $\mathbf{x}_{l+1} = \mathcal{H}_l^{res}\mathbf{x}_l + \mathcal{H}_l^{post\top}\mathcal{F}(\mathcal{H}_l^{pre}\mathbf{x}_l, \mathcal{W}_l)$。
理想情况下，相邻两个子层之间的残差变换是一个映射。
这个映射的输入是上一子层的残差流 $\mathbf{x}_{l-1}$ 与子层输出 $\mathbf{y}_{l-1} = \mathcal{F}(\hat{\mathbf{x}}_{l-1}, \mathcal{W}_{l-1})$，输出是本层残差流 $\mathbf{x}_l$ 与本层子层输入 $\hat{\mathbf{x}}_l = \mathcal{H}_l^{pre}\mathbf{x}_l$。
用激活访存量（activation memory traffic）衡量这次变换的开销，即每个 token 读、写的激活元素数。理想映射读入前两者、写出后两者，读 / 写各 $(n+1)d$，下界是 $(2n+2)d$。
DeepSeek V4 受数据依赖限制，把这个映射拆成三个串行 kernel：

$$\mathbf{x}_l = \mathcal{H}_{l-1}^{res}\mathbf{x}_{l-1} + \mathcal{H}_{l-1}^{post\top}\mathbf{y}_{l-1}, \quad (\mathcal{H}_l^{pre}, \mathcal{H}_l^{post}, \mathcal{H}_l^{res}) = \mathcal{H}(\mathbf{x}_l), \quad \hat{\mathbf{x}}_l = \mathcal{H}_l^{pre}\mathbf{x}_l$$

- 第一个 kernel 是残差更新，即 $\mathcal{F}_{\text{post,res}}$，读 $(n+1)d$、写 nd。
- 第二个 kernel 是系数预测，要在全部 nd 个元素上归约，读 nd。
- 第三个 kernel 是输入混合，即 $\mathcal{F}_{\text{pre}}$，在各条流上归约，读 nd、写 d。

再算上子层 pre-norm 读写各 d，合计 $(4n+4)d$，为下界 $(2n+2)d$ 的两倍。

实现： 把读所用的系数错开（位移）一个子层，每个子层使用上一子层预测的 $\mathcal{H}^{pre}$：

$$\mathbf{x}_{l+1} = \mathcal{H}_l^{res}\mathbf{x}_l + \mathcal{H}_l^{post\top}\mathcal{F}(\mathcal{H}_{l-1}^{pre}\mathbf{x}_l, \mathcal{W}_l), \quad (\mathcal{H}_l^{pre}, \mathcal{H}_l^{post}, \mathcal{H}_l^{res}) = \mathcal{H}(\mathbf{x}_l)$$

其中 $\mathcal{H}_{l-1}^{pre}$ 由上一子层的残差流预测，本子层开始计算之前已就绪，本子层预测出的 $\mathcal{H}_l^{pre}$ 留给下一子层，并且 V4.1 认为这一位移的性能退化可忽略。

预训练仍用多 kernel 实现，推理部署侧把残差更新、输入混合、系数预测融进单个 Mega-mHC kernel，达到 $(n+1)d$ 读 / $(n+1)d$ 写的理论值。位移在 `Block.forward` 的实现：

4.2 FP4 Main KV Cache

动机： DeepSeek V4 已经对 indexer 的 Q 与 K 做 FP4 QAT，但是 main KV 还是 FP8，而长程 agent 负载下 per-request 的 KV 体积直接决定并发。

实现： 采用 OCP 标准的 MXFP4（E2M1 + 每 16 通道一个 E4M3 scale），去掉 NVFP4 的二级全局 scale。

4.3 DSpark

V4.1-Flash 使用的 Speculative Decoding 模块是 DSpark。DSpark 具体技术参考 从 Speculative 到 DSpark：拆解 DeepSeek 推测解码、推测解码对比解析：DeepSeek-DSpark / 阶跃-JetSpec。

DSpark 同时改进了推测解码（Speculative Decoding）的 **Drafting** 和 **Verification** 两侧的效率：
Drafting：用**半自回归生成**平衡了并行 drafter（DFlash）的低延迟 & 自回归 drafter 的高接受率。
Verification：训练**置信 `head`**预测每一个位置的接受率、结合**硬件感知的调度算法**截断验证长度。
两者共同改进了推测解码的效率，即有效 token 的输出延迟。

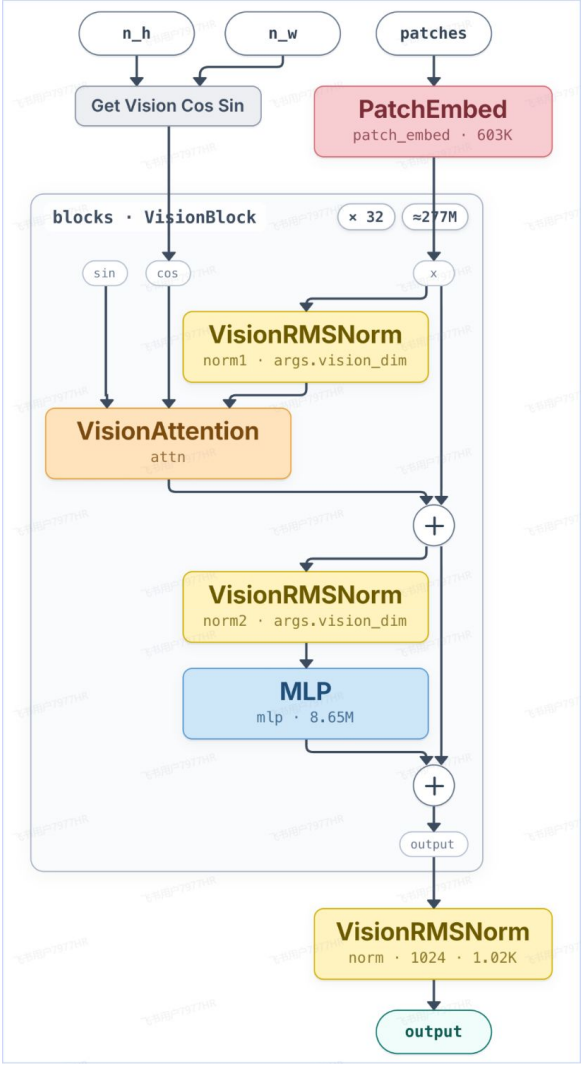
训练设置与 DeepSeek V3 的 MTP 相反：MTP 在预训练全程与主干联合训练，DSpark 则在预训练之后单独阶段训练、主干冻结。后训练阶段两者同时训练但不向主干回传梯度，既能加速在线服务也能加速 RL 与 OPD 的 rollout 生成，相关配置对应到 `config.json`：

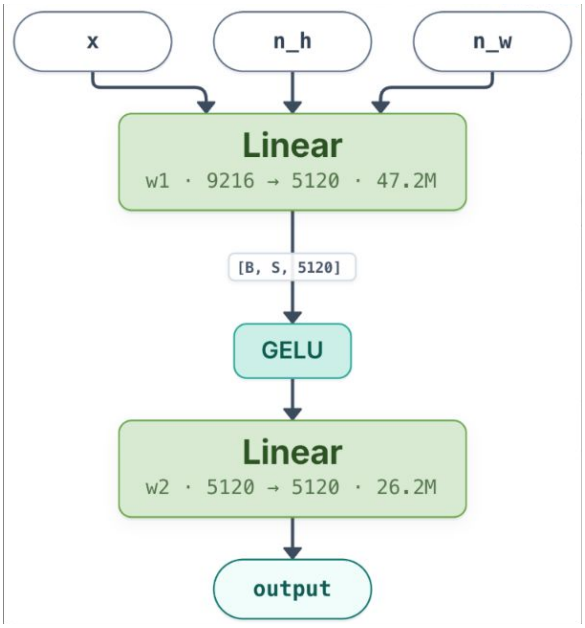
4.4 DeepSeek-ViT / 多模态负载均衡

Vision Encoder 是从头训练的 DeepSeek-ViT：32 层、隐藏维度 1024、16 head、patch 14。相对标准 ViT(2020, Google Research) 的三处改动：

- 绝对位置编码换成 2D-RoPE 以支持任意分辨率。
- Patch Embedding 的卷积换成线性投影以兼容 Muon。
- 归一化与激活改用 RMSNorm 与 SwiGLU。

比较值得注意的是 Projection 使用 3×3 的 pixel-unshuffle，相比现在常用的 2×2 有更高的压缩比：





Aligner 架构

- `downsample_ratio` 取 3，`F.unfold` 把 3×3 邻域展开到通道维度。
- 投影器是两层 MLP，本质上就是 面试官：MLLM 有哪些图像 token 压缩方法 里 pixel-shuffle 类操作。
- ViT 内 2D-RoPE 的实现参考 2D-RoPE。

语言主干模型 MoE 侧为多模态单独做了改动：

动机： 图像与文本 token 的表示分布与路由偏好都不同，把两者的负载合起来均衡会掩盖模态内部的不均衡。

实现： 把 auxiliary-loss-free 的 correction bias 拆成两套，文本与图像单独做专家选择，路由权重仍取自未加 bias 的原始分数，两套按各自负载独立更新。

4.5 Optimizer

- Head-wise Muon：权重按 head 切开再各自更新，以处理 head 之间的异质性，细节参考 Muon 优化器。
- Engram 改成动量更新 + Sinkhorn balancing：把 Muon 里的 Newton-Schulz 正交化换成对更新矩阵做行列交替 RMSNorm（迭代次数 $K = 11$ 步）。